Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)
Sector No. 26, Pradhikaran, Nigdi, Pune - 411 044
An Autonomous Institute
Affiliated to Savitribai Phule Pune University (SPPU)

DEPARTMENT OF INFORMATION TECHNOLOGY

2026-27



A Report On

PREVENTING DOUBLE BOOKING IN A RAILWAY RESERVATION SYSTEM

RailReserve: A Distributed Systems Mini Project

By

Tony Prajapati: 123B1F079
Vedant Patarwale: 123B1F069
Pratik Rajput: 123B1F135
Ishan Toraskar: 123B1F119

B.TECH (IT)

Abstract

Indian Railways sells reserved seats through one of the busiest digital ticketing platforms in the world. The hard problem at this scale is not the booking form; it is coordination. When a large burst of buyers targets the same seat, an implementation that checks availability and then writes the booking in a separate step can allow more than one buyer to pass the check before either writes. The result is a double booking, a refund, a support ticket and a loss of trust.

RailReserve is a small distributed system that isolates this problem and proves a single invariant: **exactly one active hold may exist for a seat, and a paid hold may produce exactly one booking and one payment.** Two stateless FastAPI replicas sit behind Nginx and share a Postgres 16 primary, which is the authority for inventory. A deliberately naive read-delay-write endpoint demonstrates the race. The safe path resolves it with one conditional `UPDATE` statement plus a partial unique index on active holds. Payment is idempotent through unique idempotency keys, and a hold TTL plus a background sweeper returns abandoned seats to inventory. Redis is used only for fail-open per-user rate limiting and is deliberately kept off the booking critical path.

The implementation is supported by fifteen automated tests and a six-scenario command-line driver. In the evaluated run, twenty concurrent safe hold attempts produced one `201 Created` and nineteen `409 Conflict` responses. The naive path produced twenty winners in the command-line run, while the browser showed six winners because of its per-host connection limit. Two hundred attempts across fifty seats produced fifty winners on fifty distinct seats. Ten concurrent payments with one idempotency key produced one booking and one payment. A forced-expired hold was reclaimed, and Nginx continued to serve health and booking requests when one API replica was stopped. The report maps each claim to its evidence and states the limits of the demonstration.

Keywords: distributed systems, concurrency control, double booking, idempotency, railway reservation, Postgres row locking, fault tolerance, Nginx.

Contents

Abstract 2
1. Introduction 5
1.1 Background and motivation 5
1.2 Project objectives 5
2. Problem statement and scope 6
2.1 The double-booking race 6
2.2 Problem statement 6
2.3 Proposed solution 7
2.4 Functional requirements 7
2.5 Non-functional requirements 7
2.6 Scope and limitations 8
3. System architecture and design 8
3.1 Architecture overview 8
3.2 Database model and invariants 9
3.3 The safe hold protocol 10
3.4 Payment idempotency 11
3.5 Hold expiry and inventory reclamation 11
3.6 Availability and failover 11
3.7 Consistency trade-offs 12
4. Implementation and technology 13
4.1 Technology stack 13
4.2 Implementation modules 13
4.3 HTTP API 14
4.4 Key implementation details 14
4.5 Reproducibility 14
5. Demonstration and evidence 15
5.1 Demonstration method 15
5.2 Safe race: one winner 15
5.3 Naive race: the control 16
5.4 Idempotent payment 17
5.5 Inventory sweep and expiry 18
5.6 API-tier failover 19
5.7 Six-scenario CLI evidence 19
6. Testing and results 20
6.1 Test strategy 20
6.2 Test inventory 21
6.3 Automated test evidence 21
6.4 Result summary 22
6.5 Threats to validity 22
7. SDG mapping 23
8. Advantages and applications 23
9. Limitations and future scope 23
10. Conclusion 24
References 25
Appendix A: Representative API responses 25
Appendix B: Reproducibility commands 26
Appendix C: Design review questions 27

List of Figures

Figure 1. Check-then-act race versus an atomic conditional update. The naive path lets both requests believe they won; the safe path makes Postgres the arbiter. 6
Figure 2. RailReserve logical architecture. Postgres is the authority; Redis is a fail-open side service, not part of the booking critical path. 8
Figure 3. RailReserve schema and guard chain. Each transition is protected by a database constraint, not only by application code. 9
Figure 4. Dashboard after the safe race: one winner, nineteen rejected holds, and one held seat. 16
Figure 5. Dashboard after the naive race: multiple winners for one seat. The browser produced six overlapping winners; the CLI produces twenty. 17
Figure 6. Dashboard after paying the winner twice with the same key: the second payment replays the original booking and PNR. 18
Figure 7. Six-scenario CLI evidence. The final line reports 6/6 scenarios passed. 19
Figure 8. Pytest output from the final run: fifteen tests passed. 22

List of Tables

Table 1. Public IRCTC figures used for context. These figures describe the real system, not this demonstration. 5
Table 2. Functional requirements. 7
Table 3. Non-functional requirements and acceptance criteria. 7
Table 4. Components and responsibilities. 9
Table 5. Database constraints and the failure each one prevents. 10
Table 6. Design alternatives for the seat-allocation critical section. 12
Table 7. Technology stack. 13
Table 8. Implementation modules and their responsibilities. 13
Table 9. HTTP API surface. 14
Table 10. Runtime configuration used by the demo. 15
Table 11. Demonstration scenarios and observed evidence. 19
Table 12. Automated test inventory: fifteen tests, all passing. 21
Table 13. SDG mapping and the boundary of the claim. 23

Abbreviations

API	Application Programming Interface
CP	Consistency and Partition tolerance (in the CAP trade-off)
DB	Database
HTTP	Hypertext Transfer Protocol
NFR	Non-functional requirement
PNR	Passenger Name Record
PRS	Passenger Reservation System
SQL	Structured Query Language
TTL	Time to live
UUID	Universally Unique Identifier

1. Introduction

1.1 Background and motivation

Indian Railways' reserved-ticket system is one of the largest online booking platforms in the world. IRCTC's published FY 2024-25 highlights report a record 30,155 tickets booked in one minute on 12 March 2025, an average of 13.88 lakh tickets per day, and an 86.38% online share of reserved tickets [1], [2]. The Ministry of Railways is moving to a next-generation Passenger Reservation System (PRS) designed for more than 1.5 lakh bookings per minute and 40 lakh enquiries per minute, compared with about 32,000 bookings and 4 lakh enquiries per minute in the existing setup [3]. Public reporting of the upgraded platform records 31,814 tickets in one minute on 22 May 2025 [5]. Tatkal booking opens at 10:00 for AC classes and 11:00 for non-AC classes, concentrating demand into two sharp bursts [4].

Table 1. *Public IRCTC figures used for context. These figures describe the real system, not this demonstration.*

Figure	Published value	Source
Record online bookings in one minute	30,155 on 12 March 2025; 31,814 on 22 May 2025 after the platform upgrade	[1], [5]
Average online tickets per day	13.88 lakh in FY 2024-25	[1], [2]
Online share of reserved tickets	86.38% in FY 2024-25	[1], [2]
Next-generation PRS target	More than 1.5 lakh bookings per minute and 40 lakh enquiries per minute	[3]
Tatkal opening time	10:00 for AC classes; 11:00 for non-AC classes	[4]

These numbers matter because they turn a familiar booking form into a distributed-systems problem. Multiple users, multiple application replicas and a shared inventory row must agree on one fact: who owns the seat. A booking flow that checks availability in one step and writes in another can let two buyers pass the check before either writes. The result is a double booking, a refund, a support ticket and lost confidence in the platform.

This project does not attempt to reproduce IRCTC. It isolates the smallest invariant that makes the case study technically meaningful - **one active hold per seat** - and builds enough distributed infrastructure around it to show how the invariant is protected under concurrency and node failure. The scale is deliberately small so that every claim can be reproduced on one laptop and inspected at the database level.

1.2 Project objectives

- Reproduce the check-then-act double-booking race with a controlled naive endpoint.
- Prevent the race with an atomic database guard rather than an application-level `if` check.
- Make payment safe to retry by giving every payment request an idempotency key.
- Return abandoned inventory automatically through a hold TTL and a background sweeper.
- Keep the API tier available when one of two replicas fails.
- Provide reproducible evidence through automated tests, a command-line driver and a visual dashboard.
- Explain the design trade-offs, limitations and realistic production extensions.

2. Problem statement and scope

2.1 The double-booking race

A seat hold is a classic check-then-act sequence. The application reads the seat, decides that it is available, performs some work (for example, waiting for the user to complete checkout) and then writes the booking. When two requests run at the same time, both can read `available` before either writes. The delay does not create the bug; it only widens the window in which the bug is easy to observe. The real defect is that the check and the write are separate operations with no shared atomic boundary.

A process-local mutex cannot fix this once the application has more than one replica. Each replica has its own memory, so two replicas can each take their own lock and both proceed. A correct solution must coordinate at a point that all writers share: the database row, a distributed lock service, or a serialized queue. RailReserve chooses the database because it already holds the authoritative inventory and can make the decision durable in the same transaction.

The naive path in this project deliberately performs the read, sleeps for a configurable window (`NAIVE_DELAY_MS`, set to 300 ms in Compose) and then inserts a record and marks the seat booked without a guard. It is a **teaching control**, not a product feature. The safe path performs one conditional `UPDATE`; Postgres takes a row lock, the second writer waits, and when it re-evaluates the predicate the seat is no longer available. Figure 1 contrasts the two timelines.

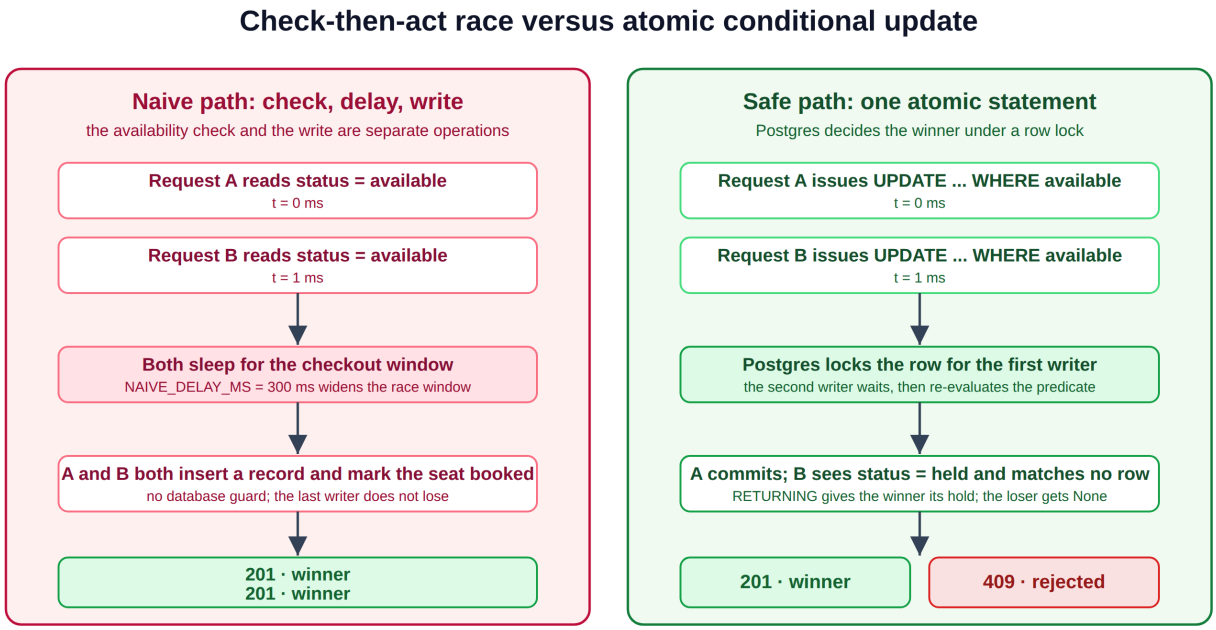


Figure 1. *Check-then-act race versus an atomic conditional update. The naive path lets both requests believe they won; the safe path makes Postgres the arbiter.*

2.2 Problem statement

Given **N concurrent purchase attempts for the same seat**, the system must ensure that **at most one active hold and at most one booking commit**, while keeping the response predictable, making payment retries safe, releasing abandoned inventory, and preserving service when an API replica fails. The system must also expose enough evidence to show that the guarantee holds under a real concurrent workload rather than only in a single-threaded test.

2.3 Proposed solution

RailReserve is a containerized, two-replica booking service. The browser dashboard talks to Nginx, which distributes requests across two FastAPI replicas. Both replicas use the same Postgres primary. The safe hold endpoint runs one conditional `UPDATE` that changes the seat from `available` (or an expired `held`) to `held`, returning the row only to the winner. A partial unique index on `holds (seat_id) WHERE status = 'active'` is a second, schema-level guard. Payment inserts a booking, a payment and the seat transition in one transaction, with unique idempotency keys. A sweeper releases expired holds, and Nginx retries a failed upstream so one dead replica does not take the service down.

The naive endpoint is kept beside the safe endpoint so that the two behaviours can be compared in the same deployment. The dashboard and the CLI driver use the same HTTP API, so the visual demonstration and the automated evidence are testing the same code path.

2.4 Functional requirements

Table 2. *Functional requirements.*

ID	Requirement	Expected behaviour
FR-01	Demo reset	Seed a train/date with a requested number of seats and clear previous demo state.
FR-02	Seat listing	Return each seat's number and status for a train and journey date.
FR-03	Safe hold	Create a hold only if the seat is available or its previous hold has expired.
FR-04	Conflict response	Reject a competing hold for the same seat with HTTP 409 and a stable error code.
FR-05	Idempotent payment	Replaying the same idempotency key returns the original booking and PNR.
FR-06	Booking retrieval	Return booking status, seat number and PNR for a booking identifier.
FR-07	Hold expiry	Expire stale holds and return the seat to available inventory.
FR-08	Naive comparison	Expose an unguarded read-delay-write hold endpoint for the race demonstration.
FR-09	Stats and health	Expose seat counts, winner counts and a liveness endpoint.

2.5 Non-functional requirements

Table 3. *Non-functional requirements and acceptance criteria.*

ID	Requirement	Acceptance criterion
NFR-01	Correctness under concurrency	Twenty concurrent safe holds for one seat produce exactly one 201 and nineteen 409 responses.
NFR-02	Retry safety	Ten concurrent payments with one key produce one booking and one payment row.
NFR-03	API availability	With one API replica stopped, health and booking requests are still served.
NFR-04	Reproducibility	The stack starts with one Docker Compose command; the test and demo commands are documented.
NFR-05	Observability	Health, stats, the event log and the `X-Upstream` response header expose useful runtime state.
NFR-06	Bounded resource use	A hold has a finite TTL; a crashed or abandoned checkout does not lock inventory forever.
NFR-07	Scope-appropriate security	The demo runs in a trusted local environment and does not claim production authentication.

2.6 Scope and limitations

The verified scope is one train (`12951`), one journey date (`2026-11-01`), a configurable seat map (fifty seats by default), two API replicas, one Postgres primary, one Redis instance and one Nginx proxy, all on one Docker host. The system demonstrates the booking invariant, idempotent payment, hold expiry and API-tier failover. It does **not** include a real payment gateway, user authentication, waitlists, quotas, Tatkal rules, notifications, multi-region replication, or production-scale load testing.

The single Postgres primary is the most important remaining single point of failure. It is acceptable for a laboratory demonstration because it makes the consistency boundary easy to inspect, but a production design would need replication, failover and partitioning. The naive endpoint exists only to make the race visible; it is never part of the safe booking flow. These boundaries are stated up front so that the evidence is not confused with a production-scale claim.

3. System architecture and design

3.1 Architecture overview

Figure 2 shows the logical topology. The browser loads a static dashboard from Nginx and calls the API through the same origin. Nginx distributes `/api/` and `/health` requests across two FastAPI replicas. Each replica is stateless: it holds no seat state in memory and can serve any request. Postgres stores seats, holds, bookings and payments and is the only component that decides whether a seat can be held. Redis stores fixed-window rate-limit counters and is allowed to fail open, because a temporary rate-limit outage must not stop legitimate bookings.

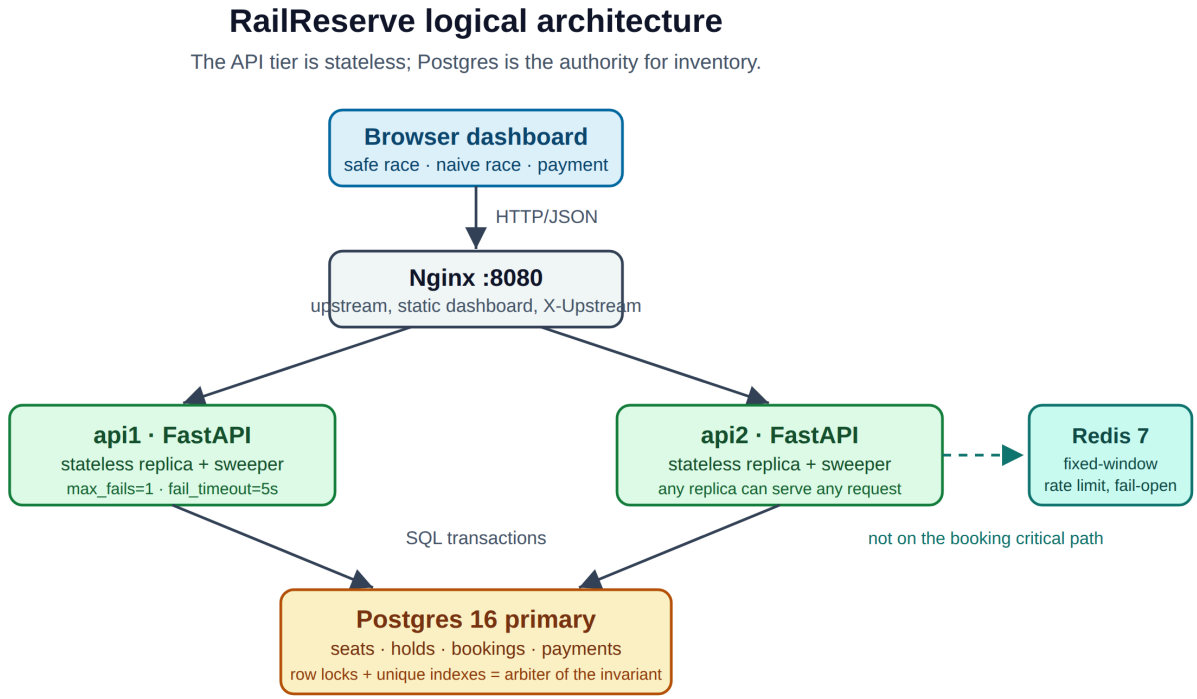


Figure 2. *RailReserve logical architecture. Postgres is the authority; Redis is a fail-open side service, not part of the booking critical path.*

Table 4. *Components and responsibilities.*

Layer	Component	Responsibility
Presentation	Dashboard (HTML/CSS/JS)	Starts safe and naive races, displays the seat map, counters and event log.
Edge	Nginx 1.27	Serves static files, proxies API requests, retries a failed upstream and exposes `X-Upstream`.
API	api1 and api2 (FastAPI)	Validate requests, run booking use cases, enforce rate limits and run the expiry sweeper.
Data	Postgres 16 primary	Stores inventory and transactions; row locks and constraints enforce the invariant.
Side service	Redis 7	Fixed-window per-user rate limiting; fails open and is never consulted for the seat decision.
Driver	scripts/demo.py	Runs six scenarios with a thread barrier and prints PASS/FAIL evidence.
Tests	pytest suite	Exercises concurrency, API, payment, expiry, reset and rate-limit behaviour.
Operations	Docker Compose	Builds the images and starts the five-service topology on one host.

The expiry sweeper runs as a daemon thread inside each API replica. Running it in both replicas is safe because the expiry statements are idempotent: they update only rows whose timestamps are already in the past. The sweep interval is one second in Compose, while the hold TTL is 120 seconds. The default application TTL is 300 seconds; the shorter Compose value keeps the live demonstration practical.

3.2 Database model and invariants

The schema is intentionally small but carries most of the correctness argument. `seats` is the inventory row. `holds` records an intent to buy and has an expiry timestamp. `bookings` records a confirmed purchase and owns the PNR. `payments` records the captured payment. `naive_bookings` is a parallel table without uniqueness constraints so the unsafe path can create multiple records for one seat. Figure 3 shows the relationships and the guard chain.

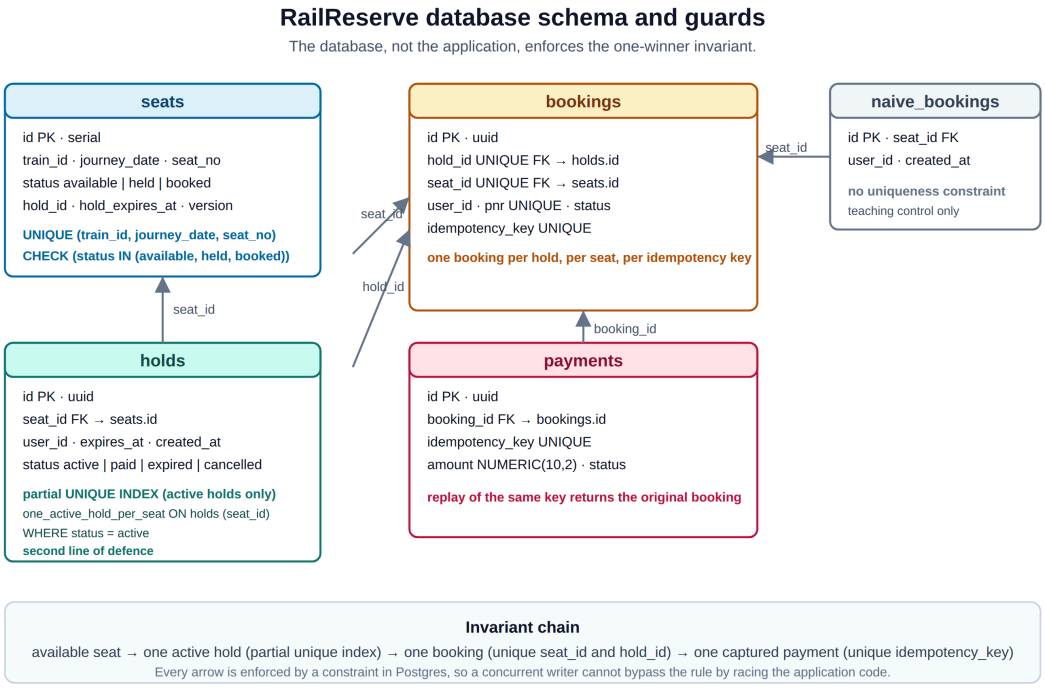


Figure 3. *RailReserve schema and guard chain. Each transition is protected by a database constraint, not only by application code.*

Table 5. *Database constraints and the failure each one prevents.*

Constraint	Table	Failure prevented
`UNIQUE (train_id, journey_date, seat_no)`	seats	Duplicate inventory rows for one physical seat.
`CHECK (status IN ('available','held','booked'))`	seats	Invalid seat states entering the inventory.
`one_active_hold_per_seat` (partial unique index)	holds	Two active holds for one seat, even if a future code path skips the conditional update.
`UNIQUE (hold_id)`	bookings	Two bookings created from one paid hold.
`UNIQUE (seat_id)`	bookings	Two confirmed bookings for one seat.
`UNIQUE (pnr)`	bookings	Duplicate PNRs.
`UNIQUE (idempotency_key)`	bookings and payments	Duplicate booking or charge when a client retries.
No uniqueness guard	naive_bookings	Intentionally absent: the naive path must be able to create multiple winners.

3.3 The safe hold protocol

The core of the safe path is a single statement. The application does not read the seat first. It asks Postgres to update the seat only if the current row matches the allowed predicate, and it uses `RETURNING` to learn whether it won. The predicate accepts an available seat or a held seat whose expiry is in the past, so an abandoned hold can be reclaimed without waiting for the sweeper. The statement also increments a version column, which is useful for diagnostics even though the demo does not rely on optimistic retries.

Listing 1. Atomic seat hold in Postgres.

```
UPDATE seats
SET status = 'held',
    hold_id = %s,
    hold_expires_at = now() + make_interval(secs => %s),
    version = version + 1
WHERE train_id = %s
  AND journey_date = %s
  AND seat_no = %s
  AND (status = 'available'
       OR (status = 'held' AND hold_expires_at < now()))
RETURNING id, seat_no, hold_expires_at;
```

Under Postgres read-committed isolation, a second `UPDATE` that targets the same row waits for the first transaction to finish. When the first transaction commits, the second re-evaluates its search condition against the updated row [6], [7]. Since the row is now `held` and its expiry is in the future, the predicate is false and no row is returned. The application maps that outcome to `409 Conflict`. The winner inserts the corresponding row into `holds` in the same transaction, so the seat and the hold record cannot diverge.

The partial unique index is a second line of defence. The application can be changed, a new endpoint can be added, or a future bug can bypass the conditional statement; the index still refuses to allow two active holds for the same seat. This is a deliberate defence-in-depth choice: the application owns the normal path, and the schema owns the invariant.

Listing 2. Schema-level guard for active holds.

```
CREATE UNIQUE INDEX one_active_hold_per_seat
    ON holds (seat_id)
    WHERE status = 'active';
```

3.4 Payment idempotency

A payment request can be retried by a user, a browser, a load balancer or a background job. Without an idempotency key, a retry can create a second booking or a second charge. RailReserve requires an `idempotency_key` on every payment. The key is unique in both `bookings` and `payments`. The payment transaction first looks up an existing booking with the same key and returns it with `replayed: true`; otherwise it validates the hold, locks the seat row, inserts the booking and payment, marks the seat `booked` and marks the hold `paid`.

Concurrent requests with the same key can still collide at the unique index. The implementation handles that case by re-reading the committed booking after the collision, using either the idempotency key or the hold identifier, and returning the original booking instead of surfacing an error. This behaviour was added after an early version returned `410 Gone` to losing concurrent payments; the regression test `test_concurrent_payment_with_same_key_creates_one_payment` now covers ten simultaneous payers.

Listing 3. Payment transaction outline with replay handling (simplified).

```
with conn.transaction():
    existing = lookup_booking_by_key(conn, idempotency_key)
    if existing:
        return replay(existing)

    hold = lock_and_validate_hold(conn, hold_id)
    seat = lock_seat(conn, hold.seat_id)
    if seat.status != 'held' or seat.hold_id != hold_id:
        existing = lookup_booking_by_key_or_hold(conn, key, hold_id)
        if existing:
            return replay(existing)
        raise HoldExpired

    booking = insert_booking(conn, hold, idempotency_key)
    insert_payment(conn, booking, amount)
    mark_seat_booked(conn, seat)
    mark_hold_paid(conn, hold)
    return booking
```

3.5 Hold expiry and inventory reclamation

A hold is a temporary reservation, not a sale. If the user abandons checkout, the seat must return to inventory. Each hold carries an `expires_at` timestamp. The sweeper marks expired active holds as `expired` and returns the corresponding seats to `available`. The safe hold statement also reclaims a stale hold inline, so a seat is not stuck until the next sweep. Payment checks the hold expiry and the seat ownership; a payment after expiry is rejected with `410 Gone`. The Compose deployment uses a 120-second TTL and a one-second sweep interval to make the behaviour observable during a presentation.

3.6 Availability and failover

The API tier is stateless, so Nginx can send any request to either replica. The upstream uses `max_fails=1` and `fail_timeout=5s`, and it retries on connection errors, timeouts and selected gateway responses. A response header, `X-Upstream`, exposes which replica served the request, which makes failover visible in a

terminal or browser. The Nginx health check uses `127.0.0.1` rather than `localhost` because the container's `localhost` can resolve to IPv6 (`::1`) while the health probe is listening on IPv4; this was a small but real deployment bug found during testing.

Listing 4. Nginx upstream and retry policy.

```
upstream api_upstream {
    server api1:8000 max_fails=1 fail_timeout=5s;
    server api2:8000 max_fails=1 fail_timeout=5s;
}

location /api/ {
    proxy_pass http://api_upstream;
    proxy_next_upstream error timeout http_502 http_503 http_504;
    proxy_connect_timeout 2s;
    proxy_read_timeout 30s;
    add_header X-Upstream $upstream_addr always;
}
```

The demo's failover boundary is the API tier, not the database. Stopping one API replica leaves the service available because the other replica can serve the request and Postgres remains reachable. Stopping Postgres would take the booking service down; that is an explicit limitation, not a hidden assumption. A production design would address it with a replicated database and a controlled failover mechanism.

3.7 Consistency trade-offs

The seat decision is deliberately a **CP choice**: consistency is preferred over availability for the single row that determines the winner. This does not make the whole system CP. The API tier can lose a replica and still serve reads and writes; Redis can fail and rate limiting becomes permissive rather than blocking; notifications and analytics could be eventual. The critical section is kept as small as possible: one row, one predicate, one transaction. Everything outside that boundary is allowed to be more available.

Table 6. *Design alternatives for the seat-allocation critical section.*

Approach	Strength	Cost or limitation	Verdict
Conditional UPDATE + unique index	Atomic, durable, simple to test; no extra critical-path service	Single primary write bottleneck; needs partitioning at large scale	Chosen
Redis distributed lock	Fast lock acquisition; familiar pattern	Adds a critical-path dependency; lock expiry, fencing and split-brain need care	Rejected for this scope
Optimistic versioning	No long lock; good for low contention	Retry loops and contention make checkout more complex	Viable alternative
Queue-based serialization	Good throughput and ordering; absorbs bursts	Adds latency and a new failure domain; more operational work	Future scale option
Application mutex	Simple in one process	Does not coordinate across replicas	Incorrect here

The chosen design does not claim to be the fastest possible. It claims to be the smallest design that makes the invariant explicit, enforceable in the data layer and observable in a live demonstration. At IRCTC scale the same invariant would still apply, but the inventory would be partitioned by train, date and seat range, and the database tier would need replication and failover.

4. Implementation and technology

4.1 Technology stack

Table 7. *Technology stack.*

Technology	Version	Role in RailReserve
Python	3.12.15 (container)	Application language and test runtime.
FastAPI	Latest in the project image	HTTP API, request validation and lifespan management.
psycopg	3.x	Postgres driver and connection pool.
Postgres	16	Authoritative inventory, transactions, row locks and constraints.
Redis	7	Fail-open fixed-window rate limiting.
Nginx	1.27	Reverse proxy, static dashboard, upstream retry and `X-Upstream`.
Docker Compose	Current host version	Builds and runs the five-service topology.
pytest	8.3.4	Automated API, concurrency and expiry tests.
httpx	Project dependency	Concurrent HTTP client used by the demo driver.
HTML/CSS/JavaScript	Browser-native	Static dashboard and event log.

4.2 Implementation modules

Table 8. *Implementation modules and their responsibilities.*

Path	Responsibility
app/main.py	FastAPI routes, lifespan startup, Redis client, health endpoint and sweeper thread.
app/service.py	Use cases: demo reset, safe hold, payment, booking lookup, naive hold, stats and expiry.
app/repository.py	SQL transactions: seat update, hold creation, payment, expiry, reset and stats.
app/db.py	Postgres connection pool initialization and shutdown.
app/config.py	Environment-backed settings for database, Redis, TTL, sweeper and rate limit.
app/schemas.py	Pydantic request models for reset, hold and payment.
app/ratelimit.py	Fixed-window per-user hold limiter; Redis failures fail open.
app/errors.py	Domain errors for unavailable seats, missing holds and expired holds.
db/init.sql	Schema, indexes and the PNR sequence.
dashboard/index.html	Static presentation interface and event log.
scripts/demo.py	Six-scenario concurrent evidence driver.
tests/	Fifteen tests covering API, repository, concurrency, payment, naive path and limits.

4.3 HTTP API

Table 9. *HTTP API surface.*

Method	Path	Purpose	Success
GET	/health	Liveness probe used by Compose and Nginx.	200
POST	/api/demo/reset	Clear and seed a train/date with a requested seat count.	200
GET	/api/seats	List seats and statuses for a train and date.	200
POST	/api/bookings/hold	Atomic safe hold with TTL.	201
POST	/api/bookings/{hold_id}/pay	Idempotent payment and booking creation.	200
GET	/api/bookings/{booking_id}	Fetch booking status, seat and PNR.	200
POST	/api/naive/hold	Unguarded read-delay-write control path.	201
GET	/api/stats	Seat counts plus naive and safe winner counts.	200

4.4 Key implementation details

The concurrency boundary is the repository layer. The service layer performs validation and translates domain errors into HTTP responses, but it does not hold a lock or keep seat state in memory. This keeps the critical section visible in one place: the SQL statement and its surrounding transaction. The connection pool is sized for the test and demo concurrency; the demo driver uses an HTTP client with a larger connection pool than a browser, which is why the CLI can send all twenty naive requests at once while the browser cannot.

The rate limiter is intentionally simple: a Redis `INCR` followed by an `EXPIRE` in a one-minute window, keyed by user identifier. If Redis is unavailable, the limiter fails open and the booking path continues. This is a deliberate availability choice for a demo, not a recommendation for every production system. A production limiter would need atomic Lua or a token bucket, per-IP and per-account policies, and careful treatment of shared NAT addresses.

The PNR is generated from a Postgres sequence (`pnr_seq`) with a `PNR` prefix. The sequence is reset with the demo data so the evidence is easy to read. Booking, payment and seat transition are in one transaction; if any statement fails, the transaction rolls back and no partial booking is committed. The dashboard is a static file served by Nginx; it is not compiled into the API image, so the API and the presentation layer can be changed independently.

4.5 Reproducibility

The full stack starts with `docker compose up --build -d`. Nginx listens on port 8080; Postgres is published on 55432 for inspection. The API is also reachable through Nginx at `http://localhost:8080`. Tests run against a separate Postgres database created by the test fixture, so they do not depend on the demo database state. The demo driver can be run from the host or from inside the API container; the wrapper `scripts/demo.sh` runs it inside `api1` and points `BASE_URL` at the Nginx service.

Table 10. *Runtime configuration used by the demo.*

Variable	Default (Compose)	Purpose
DATABASE_URL	postgresql://rail:rail@postgres:5432/railreserve	Postgres connection string.
REDIS_URL	redis://redis:6379/0	Redis connection string.
HOLD_TTL_SECONDS	120	Hold lifetime; application default is 300.
NAIVE_DELAY_MS	300	Race window for the naive path.
SWEEPER_INTERVAL_SECONDS	1	How often each replica reclaims expired holds.
RATE_LIMIT_PER_MINUTE	120	Per-user hold limit; zero disables it.
SWEEPER_ENABLED	true	Turns the expiry thread on or off.

The test command is `docker compose run --rm api1 pytest -v`. The demo command is `./scripts/demo.sh`, optionally piped to `tee demo-output.txt`. A clean reset is `curl -X POST http://localhost:8080/api/demo/reset` with a JSON body containing the train, date and seat count. The dashboard resets itself on load, which avoids stale state during a live presentation.

5. Demonstration and evidence

5.1 Demonstration method

The evidence comes from three sources. First, the pytest suite exercises the repository and API against a real Postgres test database. Second, the CLI driver uses a thread barrier to release all requests at the same instant and prints the observed status codes and database counts. Third, the dashboard provides a visual record of the same behaviour with a seat map and event log. The CLI is the authoritative source for concurrency counts because it controls its own connection pool; the dashboard is the presentation aid.

The demo uses a fixed train (`12951`), a fixed journey date (`2026-11-01`) and fifty seats (`S1-01` to `S1-50`). The target seat is `S1-01`. The hold TTL is 120 seconds in Compose. The naive delay is 300 milliseconds. These values are intentionally small enough to demonstrate the behaviour quickly, but the correctness argument does not depend on them: the safe path remains correct for any TTL or delay.

5.2 Safe race: one winner

The safe race resets the demo to one seat and releases twenty hold requests through a barrier. The expected result is one `201 Created`, nineteen `409 Conflict` responses, and a stats endpoint that reports `held: 1`. The observed result matches exactly: `201=1, 409=19, stats.held=1`. The dashboard shows the seat in the held state and records the race in the event log (Figure 4).

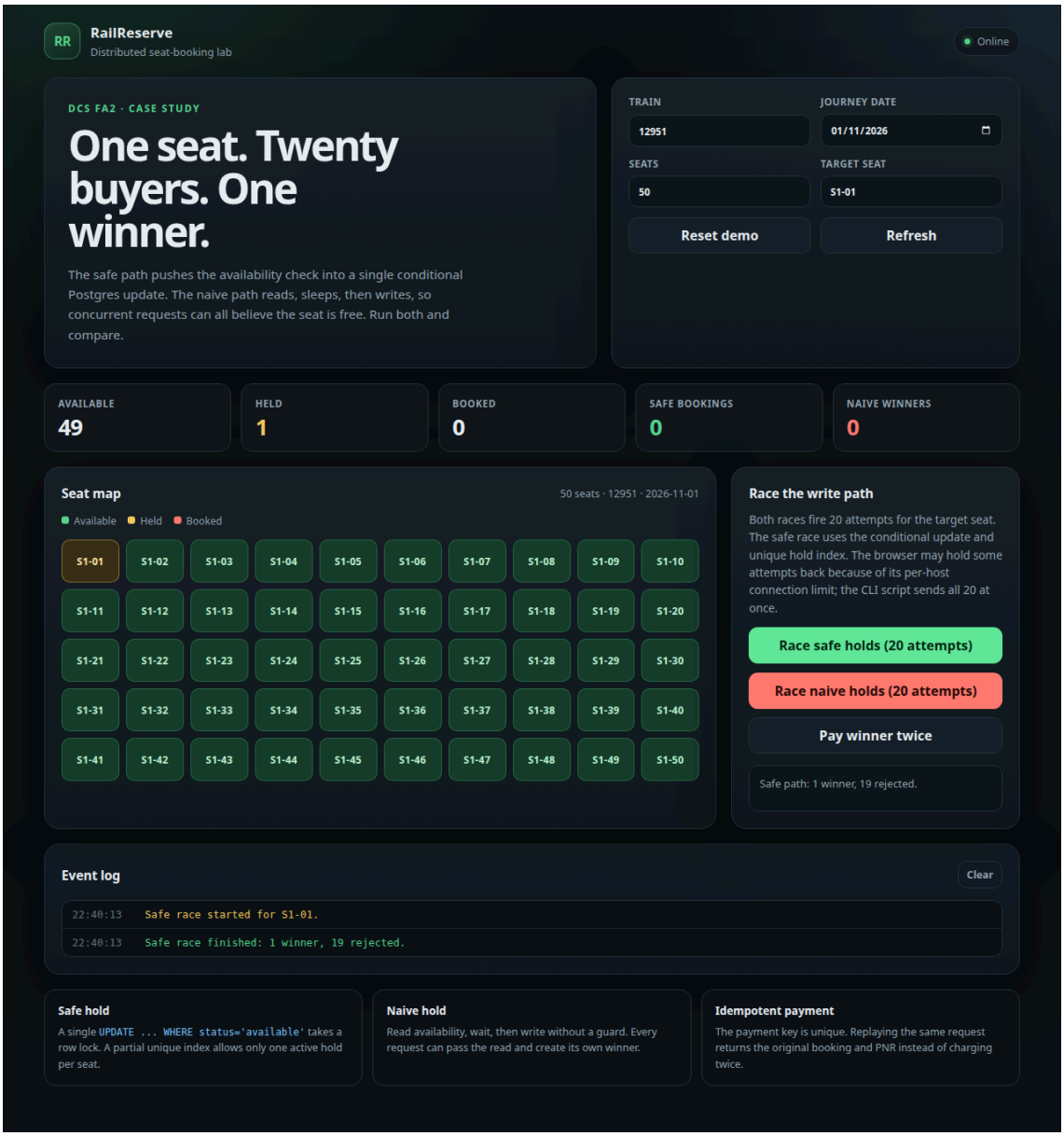


Figure 4. *Dashboard after the safe race: one winner, nineteen rejected holds, and one held seat.*

The test suite repeats the same race at the service layer with twenty threads and asserts `results.count('ok') == 1` and `results.count('rejected') == 19`. The database is checked afterwards to confirm that only one seat is held. The API-level evidence and the service-level test therefore agree, which reduces the chance that the result is an artefact of the dashboard.

5.3 Naive race: the control

The naive race resets to one seat and releases the same twenty requests against `/api/naive/hold`. In the CLI run, all twenty requests return `201` and the stats endpoint reports `naive_winners: 20`. This is the failure the safe path is designed to prevent. The browser screenshot shows six winners from twenty attempts (Figure 5). The difference is not a server inconsistency: a browser using HTTP/1.1 typically limits concurrent connections to one origin, so not all twenty requests overlap. The CLI uses an HTTPX client with a larger connection pool and sends all twenty at once. The honest conclusion is that the browser demonstrates the race, while the CLI measures its full width.

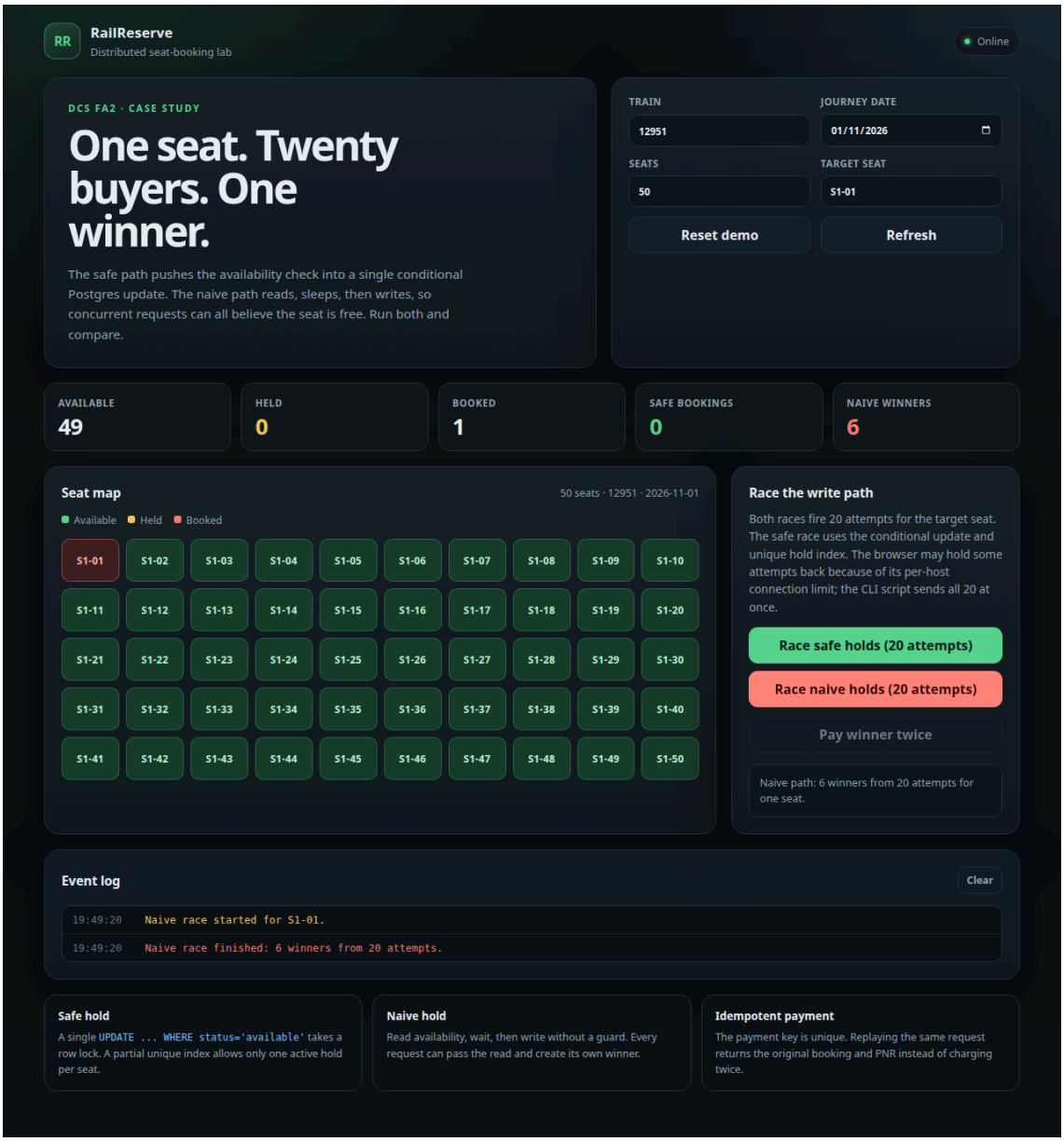


Figure 5. *Dashboard after the naive race: multiple winners for one seat. The browser produced six overlapping winners; the CLI produces twenty.*

The naive path is not a subtle bug in the safe path. It is a separate endpoint that deliberately omits the guard so the audience can see why the guard matters. Removing the naive endpoint would make the demonstration less convincing, not more correct.

5.4 Idempotent payment

The payment demonstration holds one seat and sends ten concurrent payment requests with the same idempotency key. The expected invariants are: all ten responses return `200`, all successful responses carry the same booking identifier, and the database contains exactly one payment row for the key. The observed result is `200=10, booking_ids=1, payments=1, pnr=PNR1000000000`. The dashboard's payment replay shows the same booking and PNR on the second payment (Figure 6). The second response reports `replayed: true`.

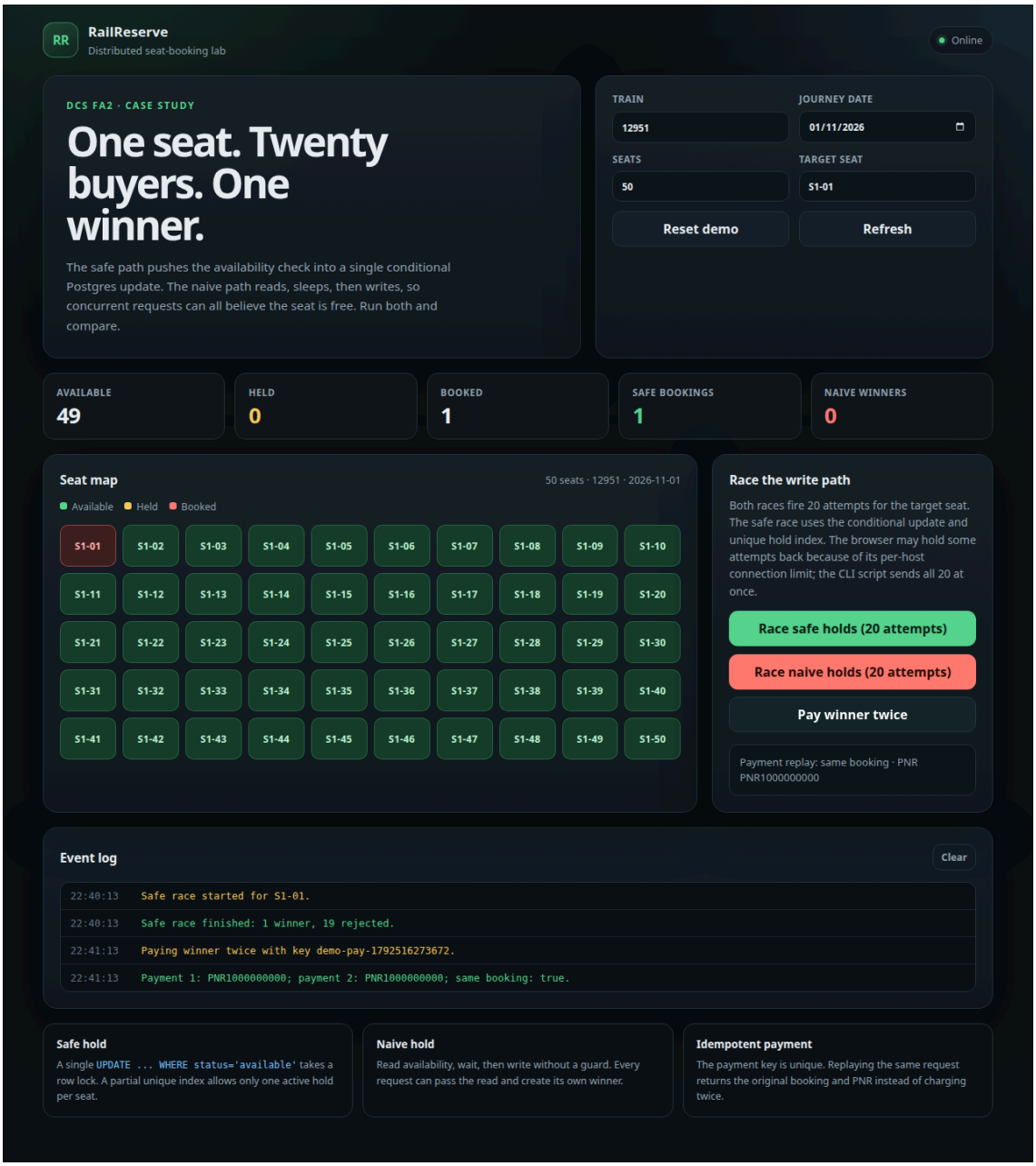


Figure 6. *Dashboard after paying the winner twice with the same key: the second payment replays the original booking and PNR.*

This test is important because retries are normal in distributed systems. A client that times out after a successful payment may retry; a load balancer may retry an idempotent request; a user may press the pay button twice. The unique key converts those retries into reads of the original result rather than new charges.

5.5 Inventory sweep and expiry

The inventory sweep resets the demo to fifty seats and sends two hundred hold attempts, four per seat. The expected outcome is fifty winners on fifty distinct seats and one hundred and fifty conflicts. The observed result is `201=50, distinct seats=50, 409=150`. This shows that the guard scales beyond a single hot seat: it protects each row independently, and the application does not need a global lock to keep the inventory consistent.

The expiry scenario creates a hold, moves both the hold expiry and the seat expiry into the past through a direct SQL update, and then attempts a second hold on the same seat. The second hold returns `201`, proving that an expired hold is reclaimable. The payment path also checks expiry: a payment after expiry is rejected with `410 Gone`. The direct SQL update is a deliberate test technique; waiting for the full TTL would add two minutes to every demo run without testing anything additional.

5.6 API-tier failover

The failover check stops `api1`, requests health and seats through Nginx, and then starts `api1` again. In the manual verification, health returned `200`, a hold for `S1-02` returned `201`, and the stats endpoint reported one held seat while `api1` was down. The `X-Upstream` response header showed which replica served the request. After `api1` was restarted, both replicas were healthy again. The automated demo driver checks health and prints the stop/start commands; it does not stop the container from inside the API container, because that would require giving the container control over the host's Docker daemon. The manual result and the printed commands are both reported, and the limitation is stated in Section 6.5.

5.7 Six-scenario CLI evidence

The CLI driver prints a stable summary for each scenario and exits non-zero if any scenario fails. The final run reports `6/6 scenarios passed` (Figure 7). The output is deliberately terse so it can be read aloud during a presentation: each line names the scenario and the counts that matter. The driver waits for `/health` through Nginx before starting; this avoids a startup race in which Nginx marks both upstreams as temporarily unavailable because the API containers are still booting.



Figure 7. *Six-scenario CLI evidence. The final line reports 6/6 scenarios passed.*

Table 11. *Demonstration scenarios and observed evidence.*

Scenario	Invariant under test	Observed result
naive_race	An unguarded path can produce multiple winners.	20/20 requests won in the CLI; 6/20 in the browser due to the per-host connection limit.
safe_race	Exactly one safe hold wins for one seat.	1 x 201, 19 x 409, stats.held = 1.
inventory_sweep	The guard works across many seats.	200 attempts: 50 winners, 50 distinct seats, 150 x 409.
idempotent_payment	A repeated key returns one booking and one payment.	10 x 200, one booking, one payment, one PNR.
expiry	An expired hold can be reclaimed.	Second hold after forced expiry returned 201.
failover	The API tier survives one replica failure.	Manual stop of api1: health 200, hold 201; api1 restarted healthy.

6. Testing and results

6.1 Test strategy

The tests run against a real Postgres database rather than a mock. The fixture creates a separate `railreserve_test` database, applies `db/init.sql`, and truncates the tables before each test. This matters because the correctness argument depends on actual row locking, unique indexes and transaction behaviour; a mocked repository would test the mock, not the database. The suite uses twenty threads and a `Barrier` to make the interleaving deterministic, and it exercises the service layer directly for the concurrency cases. API shape, reset and health are tested through FastAPI's `TestClient`. The rate limiter is tested with a small fake Redis object, so the test does not depend on a running Redis instance.

The tests are grouped by concern: concurrency, holds and expiry, payment, the naive control path, reset and stats, health, and rate limiting. A test that passes because the table happened to be empty would be misleading, so the concurrency tests reset the demo and seed exactly the rows they need. The safe race test uses real Postgres threads; the naive test replaces the sleep with a barrier so every thread completes its read before any thread writes.

6.2 Test inventory

Table 12. *Automated test inventory: fifteen tests, all passing.*

Area	Test	What it proves
Concurrency	test_safe_hold_allows_exactly_one_winner	20 threads produce exactly one winner and one held seat.
Concurrency	test_naive_path_can_produce_multiple_winners	The unguarded path can create more than one winner.
API	test_demo_reset_and_stats_endpoints	Reset, seat listing and stats work through the API.
Health	test_health	The liveness endpoint returns the expected payload.
Hold	test_hold_seat_marks_seat_held	A successful hold changes the seat status.
Hold	test_second_hold_for_same_seat_is_rejected	A second hold is rejected while the first is active.
Hold	test_expired_hold_can_be_reclaimed	An expired hold can be replaced by a new hold.
Naive	test_naive_hold_creates_booking	The naive path creates one booking without contention.
Naive	test_naive_hold_rejects_unavailable_seat	The naive path rejects an already booked seat.
Payment	test_payment_is_idempotent_and_marks_seat_booked	Replay returns the same booking; the seat becomes booked.
Payment	test_concurrent_payment_with_same_key_creates_one_payment	10 concurrent payers produce one booking and one payment.
Payment	test_payment_after_expiry_is_rejected	An expired hold cannot be paid.
Rate limit	test_rate_limit_allows_within_limit_and_blocks_after	The fixed window allows the limit and blocks the next request.
Repository	test_reset_demo_seeds_requested_seats	Reset creates the requested seats in order, all available.
Repository	test_reset_demo_clears_previous_state	Reset removes previous state and restarts identifiers.

6.3 Automated test evidence

The final run collected fifteen tests and reported `15 passed, 1 warning in 2.52s` (Figure 8). The warning is a deprecation notice from Starlette's test client about an AnyIO alias; it does not affect the result. The short duration is expected: the suite uses a local Postgres container and in-process API calls, and the concurrency tests use twenty threads rather than a large load generator. The point is deterministic correctness evidence, not throughput measurement.

```
pytest -v — 15 concurrency and API tests

============================= test session starts ==============================
platform linux -- Python 3.12.15, pytest-8.3.4, pluggy-1.7.0 -- /usr/local/bin/python3.12
cachedir: .pytest_cache
rootdir: /app
configfile: pytest.ini
plugins: anyio-4.15.1
collecting ... collected 15 items

tests/test_concurrency.py::test_safe_hold_allows_exactly_one_winner PASSED [  6%]
tests/test_concurrency.py::test_naive_path_can_produce_multiple_winners PASSED [ 13%]
tests/test_demo.py::test_demo_reset_and_stats_endpoints PASSED           [ 20%]
tests/test_health.py::test_health PASSED                                 [ 26%]
tests/test_hold.py::test_hold_seat_marks_seat_held PASSED                [ 33%]
tests/test_hold.py::test_second_hold_for_same_seat_is_rejected PASSED    [ 40%]
tests/test_hold.py::test_expired_hold_can_be_reclaimed PASSED            [ 46%]
tests/test_naive.py::test_naive_hold_creates_booking PASSED              [ 53%]
tests/test_naive.py::test_naive_hold_rejects_unavailable_seat PASSED     [ 60%]
tests/test_payment.py::test_payment_is_idempotent_and_marks_seat_booked PASSED [ 66%]
tests/test_payment.py::test_concurrent_payment_with_same_key_creates_one_payment PASSED [ 73%]
tests/test_payment.py::test_payment_after_expiry_is_rejected PASSED      [ 80%]
tests/test_ratelimit.py::test_rate_limit_allows_within_limit_and_blocks_after PASSED [ 86%]
tests/test_repository.py::test_reset_demo_seeds_requested_seats PASSED   [ 93%]
tests/test_repository.py::test_reset_demo_clears_previous_state PASSED   [100%]

============================== warnings summary ===============================
../usr/local/lib/python3.12/site-packages/starlette/testclient.py:40
  /usr/local/lib/python3.12/site-packages/starlette/testclient.py:40: DeprecationWarning: The anyio.abc.BlockingPortal alias is
deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = typing.Callable[[], typing.ContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 15 passed, 1 warning in 2.52s =========================
```

Figure 8. *Pytest output from the final run: fifteen tests passed.*

6.4 Result summary

The evidence supports four claims. First, the safe hold path allows exactly one winner under twenty-way concurrency, both in the API and in a direct service-level test. Second, the naive control path can produce multiple winners, which establishes that the race is real and that the safe path's result is not an artefact of low concurrency. Third, payment retries with a shared key produce one booking and one payment, even when ten requests arrive together. Fourth, an expired hold can be reclaimed and one API replica can fail without taking down the service.

The inventory sweep adds a broader check: the guard is not specific to one hot seat. Fifty seats and two hundred attempts produced fifty winners on fifty distinct seats, with the remaining attempts correctly rejected. This is evidence of per-row correctness at a small scale, not a throughput result. The report deliberately avoids claiming a booking rate or a latency figure because the demo does not include a load generator or production hardware.

6.5 Threats to validity

- **Browser concurrency.** The browser may serialize requests because of per-host connection limits, so the visual naive race can show fewer winners than the CLI. The CLI is the authoritative measurement.
- **Local single host.** All services run on one Docker host. The failover test covers an API replica failure, not a network partition, database failure or host failure.
- **Small scale.** Twenty concurrent requests and fifty seats are enough to demonstrate the invariant, but they are not a throughput benchmark.
- **Manual failover step.** The automated driver verifies health and prints the commands; the stop/start step was performed manually. A production test would automate the container lifecycle from outside the API container.

- **No real payment gateway.** Idempotency is demonstrated at the booking service boundary, not against a bank or payment aggregator with its own retry semantics.
- **Fixed-window limiter.** The rate limiter is intentionally simple and fails open; it is not a production-grade abuse-prevention system.

7. SDG mapping

The project maps primarily to **SDG 9: Industry, Innovation and Infrastructure**, because it demonstrates a resilient digital service pattern for a public transport system. Reliable ticketing infrastructure is part of the digital layer that keeps mass transit usable at peak demand. It also has an indirect link to **SDG 11: Sustainable Cities and Communities**, because efficient public transport depends on trustworthy digital access to seats; a booking system that double-books passengers undermines confidence in the service.

Table 13. *SDG mapping and the boundary of the claim.*

SDG	Project contribution	Boundary of the claim
SDG 9: Industry, Innovation and Infrastructure	Shows how a service can protect a critical invariant while remaining available across an API-node failure.	Educational prototype; not a production deployment or a measured infrastructure impact.
SDG 11: Sustainable Cities and Communities	Supports reliable access to public transport ticketing, which is part of making mass transit usable.	Indirect and educational; the project does not measure modal shift or emissions.

The mapping is deliberately modest. A laboratory prototype does not by itself produce economic or environmental impact. Its contribution is to make the coordination problem visible and to show a design that a real system could extend.

8. Advantages and applications

The design has several practical advantages. The invariant lives in the database, so it cannot be bypassed by adding another API replica. The API tier is stateless and can be scaled horizontally behind a load balancer. Payment retries are safe by construction. Abandoned inventory is reclaimed automatically. The dashboard and CLI make the behaviour observable without reading the source. The implementation is small enough to explain in a viva and complete enough to reproduce on a laptop.

- **Rail, bus and air ticketing:** seat or berth inventory with a short checkout window.
- **Event ticketing:** a fixed number of seats for a performance, with a high-demand opening time.
- **Appointment and slot booking:** one patient, room or service slot per time window.
- **Warehouse and logistics slots:** reserving a dock or storage position for a limited period.
- **Any inventory with a hold-then-pay flow:** the same conditional-update and idempotency pattern applies.

The project is not a template for every booking domain. Domains with complex quotas, waitlists, cancellations, fare rules and partial refunds need additional state and policy. The contribution is the core coordination pattern, not a complete product.

9. Limitations and future scope

The most important limitation is the single Postgres primary. It is the authority for inventory and therefore the remaining single point of failure. A production design would use a replicated primary with controlled failover, or a partitioned inventory service with a consensus protocol for the critical row. The current system

also does not implement authentication, authorisation, a real payment gateway, cancellation, refunds, waitlists, quotas or Tatkal-specific rules. It does not include a load generator, so it cannot make latency or throughput claims.

Future work can be grouped into four tracks. **Scale:** partition inventory by train, date and seat range; add read replicas for read-heavy endpoints; use a queue for notifications and analytics. **Reliability:** add database replication and failover; run chaos tests for network partitions, replica loss and database failover; add idempotency keys to more write endpoints. **Security:** add authenticated users, per-account rate limits, audit logging and secrets management. **Observability:** add structured logs, metrics for hold success and conflict rates, traces across Nginx and the API, and alerting on sweeper lag or database saturation.

A realistic next step is a controlled load test with a partitioned inventory and a replicated database. That would let the project move from a correctness demonstration to an evidence-based capacity study. The current report should be read as a proof of the invariant and the failure-handling pattern, not as a capacity claim.

10. Conclusion

RailReserve demonstrates that double booking is a coordination problem, not a UI problem. The project reproduces the race with a naive path, prevents it with a single conditional `UPDATE` and a partial unique index, makes payment retries safe with idempotency keys, reclaims abandoned inventory with a TTL and sweeper, and keeps the API tier available when one replica fails. The evidence is consistent across the API, the service-level tests and the CLI driver: one winner in the safe race, multiple winners in the naive control, one booking for ten concurrent payments, and a successful reclaim of an expired hold.

The broader lesson is that the invariant should live where the data lives. Application checks are necessary for good error messages, but they are not sufficient once more than one process can write. Unique constraints, row locks and transactions are the last line of defence, and they are also the easiest part of the design to test. The same principle applies far beyond railway reservations: any system that sells a finite resource needs an explicit winner, a retry-safe payment path and a way to return abandoned inventory. RailReserve is a small, reproducible demonstration of those three ideas.

References

[1] IRCTC, "Internet Ticketing," official website, accessed 20 October 2026. https://irctc.com/internet-ticketing.php

[2] IRCTC, "Annual Report 2024-25," 2025. https://irctc.com/annual-report.php

[3] Press Information Bureau, Ministry of Railways, "Four Decades of Passenger Reservation in Indian Railways," 6 August 2026. https://www.pib.gov.in/PressNoteDetails.aspx?ModuleId=3&NoteId=159450

[4] Indian Railways, "Salient Features of the Tatkal Scheme," Railway Board. https://indianrailways.gov.in/railwayboard/uploads/directorate/traffic_comm/Salient_features_of_Tatkal_Scheme.pdf

[5] The Hindu, "Railways suspends 2.5 crore user IDs suspected of fraudulent ticket booking," 4 June 2025. https://www.thehindu.com/news/national/railways-suspends-25-crore-user-ids-suspected-of-fraudulent-ticket-booking/article69657318.ece

[6] PostgreSQL Global Development Group, "Transaction Isolation," PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/transaction-iso.html

[7] PostgreSQL Global Development Group, "Explicit Locking," PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/explicit-locking.html

Appendix A: Representative API responses

The following exchanges were captured from the running stack. Identifiers and timestamps are from that run; the structure is stable. The reset call seeds three seats, the safe hold wins once, the second hold is rejected, and the first payment is replayed with the same key.

```
POST /api/bookings/hold  -> 201
{
  "hold_id": "418e3849-f49f-45b9-90b3-0f93481c9026",
  "seat_id": 1,
  "seat_no": "S1-01",
  "expires_at": "2026-10-20T17:32:40.206462Z"
}

POST /api/bookings/hold  (same seat)  -> 409
{ "detail": "seat_unavailable" }

POST /api/bookings/{hold_id}/pay  (first)  -> 200
{
  "booking_id": "fa9f7f28-17ce-4398-b958-0615ac72e8e1",
  "pnr": "PNR1000000000",
  "status": "confirmed",
  "seat_no": "S1-01",
  "replayed": false
}

POST /api/bookings/{hold_id}/pay  (replay)  -> 200
{
  "booking_id": "fa9f7f28-17ce-4398-b958-0615ac72e8e1",
  "pnr": "PNR1000000000",
  "status": "confirmed",
  "seat_no": "S1-01",
  "replayed": true
}

GET /api/stats  -> 200
{
  "available": 2, "held": 0, "booked": 1,
  "naive_winners": 0, "safe_bookings": 1
}
```

Appendix B: Reproducibility commands

The report uses the following commands. They assume Docker and Docker Compose are installed and the current directory is `railreserve`.

```
docker compose up --build -d
open http://localhost:8080

docker compose run --rm api1 pytest -v
./scripts/demo.sh | tee demo-output.txt

curl http://localhost:8080/health
curl "http://localhost:8080/api/stats?train_id=12951&journey_date=2026-11-01"

docker compose stop api1
curl -i http://localhost:8080/health
docker compose start api1
```

The project artefacts used as evidence are the source tree, `demo-output.txt`, the three dashboard screenshots and the pytest run. The test database is created automatically; no manual schema step is required. If the Postgres volume was created before the schema was added, run `docker compose down` once so that `db/init.sql` is applied on the next start.

Appendix C: Design review questions

These questions are a short preparation aid for the FA2 discussion. The answers are intentionally concise.

Why not keep inventory in Redis for speed? Redis could make the check fast, but the winner still needs an atomic operation and durable confirmation. Postgres already provides both, and it holds the authoritative record. Redis stays on the non-critical path as a fail-open rate limiter.

What happens when two payments arrive together? The unique idempotency key means only one booking and one payment can commit. A loser re-reads the committed booking by key or hold identifier and returns the same result with `replayed: true`.

What if an API replica dies mid-request? The hold transaction either commits or rolls back. If it committed, the hold exists and can be paid. If it rolled back, the seat remains available. Nginx retries the request on the surviving replica.

What if Nginx dies? Nginx is a single proxy in this demo. A production deployment would run multiple load balancers or use a managed load balancer with health checks. The stateless API replicas are already prepared for that.

Why is the naive path in the code? It is the control group. It makes the race measurable and shows that the safe path is not winning because the workload is too small to collide.

What is the isolation level? The critical statement is a single `UPDATE ... WHERE` under Postgres read-committed isolation. The row lock determines the winner; the second writer re-evaluates the predicate after the first commits.

How does this relate to IRCTC scale? The invariant is the same; the implementation would be partitioned by train, date and seat range, with a replicated database and asynchronous downstream work. The demo isolates the coordination primitive, not the full production topology.

Could a unique index alone solve the problem? The partial unique index on active holds is a strong guard, but the conditional update also gives a clean winner and a useful 409 response. Using both is defence in depth: the application owns the normal path and the schema owns the invariant.